

【实验报告】

课程名： 计算机组成与体系结构（H）

实验名称： lab3 跳转和条件跳转指令等

学生： 杨箫羽

学号： 24300240069

时间： 2026 年 4 月 21 日

一、实验内容与实现思路说明

实验 3 要求增加和跳转和条件跳转相关的指令。为了实现这些指令，我在 EX 阶段增加了 branch_cmp、redirect_pc 和 redirect_valid 的生成模块来实现 pc 的跳转，在 control_unit 中增加了 branch_type、is_baj、cmpsrc 三个控制信号，同时为了解决数据冒险，新增 load_use_hazard 模块。

在 verilator 上通过测试之后，在 vivado 上实现上板。

下面分总体电路图、新增的模块和做出重要修改的模块讲解、调试四部分进行具体说明。

Verilator:

```
Running AES correctness checks...
[TEST] AES-128 test vector Encrypt
[PASS] AES-128 test vector Encrypt passed
[TEST] AES-128 test vector Decrypt
[PASS] AES-128 test vector Decrypt passed
[PASS] AES correctness checks passed
[RNG] heavy_rng_mod result=0, time(us)=285183
[RNG] heavy_rng_mod it/s=3508
[src/cpu/cpu-exec.c:393,cpu_exec] nemu: HIT GOOD TRAP at pc = 0x0000000080000030
[src/cpu/cpu-exec.c:394,cpu_exec] trap code:0
[src/cpu/cpu-exec.c:74,monitor_statistic] host time spent = 394708 us
[src/cpu/cpu-exec.c:76,monitor_statistic] total guest instructions = 1243348
[src/cpu/cpu-exec.c:77,monitor_statistic] simulation frequency = 3150045 instr/s
Program execution has ended. To restart the program, exit NEMU and run again.
sh: 1: spike-dasm: not found
```

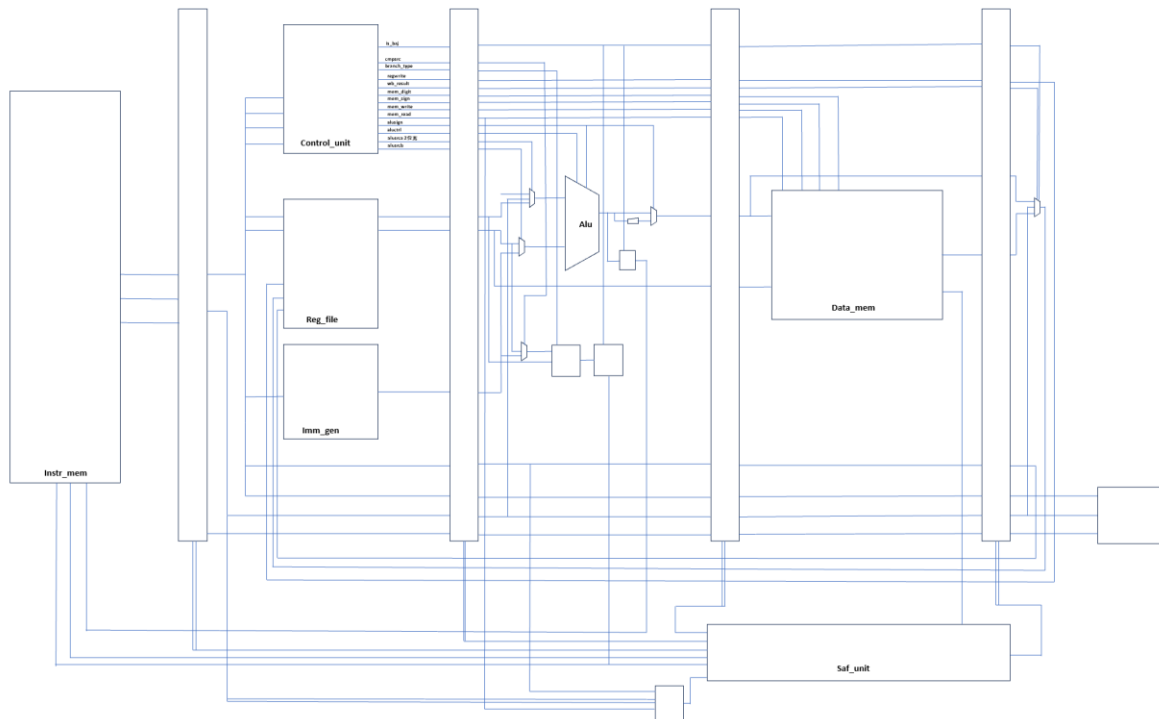
上板:

The screenshot displays the Verilator GUI interface. On the left, there is a sidebar with various settings for the terminal connection, including the port name (COM5), baud rate (9600), data bits (8), parity (None), and stop bits (1). Below these settings are checkboxes for '接收设置' (Receive Settings) and '发送设置' (Transmit Settings). The main window shows the output of the program, which includes the AES benchmark and correctness checks. The output indicates that the program passed all tests and completed the benchmark successfully.

```
2026/4/19 15:23:25 -> 正在尝试重新连接 [第 84 次] COM5 设备不存在。
2026/4/19 15:23:55 -> 正在尝试重新连接 [第 85 次] COM5 设备不存在。
2026/4/19 15:24:25 -> 正在尝试重新连接 [第 86 次] COM5 设备不存在。
2026/4/19 15:24:55 -> 正在尝试重新连接 [第 87 次] COM5 设备不存在。
2026/4/19 15:25:25 -> 正在尝试重新连接 [第 88 次] COM5 设备不存在。
2026/4/19 15:25:55 -> 正在尝试重新连接 [第 89 次] COM5 设备不存在。
2026/4/19 15:26:25 -> 正在尝试重新连接 [第 90 次] COM5 设备不存在。
2026/4/19 15:26:55 -> 正在尝试重新连接 [第 91 次] COM5 设备不存在。
2026/4/19 15:27:25 -> 正在尝试重新连接 [第 92 次] COM5 设备不存在。
2026/4/19 15:27:55 -> 正在尝试重新连接 [第 93 次] 连接成功

AES benchmark + correctness
Running AES correctness checks...
[TEST] AES-128 test vector Encrypt
[PASS] AES-128 test vector Encrypt passed
[TEST] AES-128 test vector Decrypt
[PASS] AES-128 test vector Decrypt passed
[PASS] AES correctness checks passed
[RNG] heavy_rng_mod result=0, time(us)=2027644
[RNG] heavy_rng_mod it/s=493
AES benchmark + correctness
Running AES correctness checks...
[TEST] AES-128 test vector Encrypt
[PASS] AES-128 test vector Encrypt passed
[TEST] AES-128 test vector Decrypt
[PASS] AES-128 test vector Decrypt passed
[PASS] AES correctness checks passed
[RNG] heavy_rng_mod result=0, time(us)=2027644
[RNG] heavy_rng_mod it/s=493
```

二、总体电路图



三、修改和新增的重要模块

1、redirect_pc

接口：

```
module redirect_pc_unit (  
    input  logic [63:0] alu_res,  
    input  logic [1:0] is_baj_e,  // 0:none, 1:branch, 2:jal, 3:jalr  
    output logic [63:0] redirect_pc  
);
```

功能：接收 alu 计算结果，输出 pc 跳转的结果。从 jalr 指令和其他指令两种类型进行分类讨论。当 is_baj_e 是 0、1、2 的时候正常输出 input。3 的时候输出 alu_res & ~1。

2、redirect_valid

```
module redirect_valid_unit (  
    input  logic    cmp_res,  
    input  logic [1:0] is_baj_e,  
    output logic    redirect_valid  
);
```

接口：

		sllw	srlw	sraiw	sllw	srlw	sraw	auipc	jal	jalr
opcode		0011011	0011011	0011011	0111011	0111011	0111011	0010111	1101111	1100111
funct3		011	101	101	011	101	101			000
funct7					0000000	0000000	0100000			
bit30		0	0	1	0	0	1	0	0	0
alusign	1时针对rv64i这种指令，做32->64的扩展	1	1	1	1	1	1	0	0	0
aluctrl	(0-9) +/异或/或/与/-/有符号小于/无符号小于	7	8	9	7	8	9	0	0	0
alusrc	0为0, 1为rs1val, 2为pc	1	1	1	1	1	1	2	1	1
alusrcb	0为rs2val, 1为imm	1	1	1	0	0	0	1	1	1
regwrite	0不写, 1写	1	1	1	1	1	1	1	1	1
mem_write	0不写, 1写	0	0	0	0	0	0	0	0	0
mem_read	0不读, 1读	0	0	0	0	0	0	0	0	0
mem_sign	0时符号扩展, 1时零扩展	0	0	0	0	0	0	0	0	0
mem_digit	8/16/32/64	0	0	0	0	0	0	0	0	0
wb_result	0为alu结果, 1为mem读结果, 2为pc+4	0	0	0	0	0	0	0	2	2
branch_type	0-5分别代表=, !=, 有符号<, 有符号>=, 有符号<=, 有符号>	0	0	0	0	0	0	0	0	0
cmpsrc	0为rs2val, 1为imm	0	0	0	0	0	0	0	0	0
is_baj	0代表都不是, 1代表b类, 2代表jal, 3代表ja	0	0	0	0	0	0	0	2	3

四、调试

1、一开始 Lab2 在随机测试中出现以为 load-use 冒险错误，日志显示前一条 load 已经把 t3 写成正值，但下一条 addi t3,t3,imm 结果只剩立即数，说明相关数据没有被使用。最初检查了 EX 级 forwarding，增加了 load_use 模块。但是增加之后还是错误。后来结合波形发现：当 addi 进入 EX 时，load 已经离开 WB，而 addi 前一拍仍在 ID，因此真正缺失的是 WB→ID 同拍写后读旁路，不是 WB→EX。最后在 reg_file.sv 的组合读口加入判断：若 regwrite&&rd==rs1/rs2，直接输出 writedata，问题解决。

2、进入 Lab3 后，运行时报错访问 0x20003000 out of bound。查阅 wiki 后发现这是外设 MMIO 地址空间，DiffTest 需要对这类访存跳过检查，否则会误判。最初直接照抄 wiki 中 .skip(mem & memaddr[31]==0)，但工程内并没有这两个信号名，导致编译失败。后来在 datapath 中新增提交级信号：当前提交指令是否为 load/store，以及其访存地址；再传到 core.sv，改为使用真实信号控制 .skip(...)，问题解决。

3、还出现过端口列表语法错误和位宽错误。原因是在 datapath/core 中新增输出端口时，端口名不统一、漏写位宽声明。统一命名并显式声明后恢复正常。

4、vivado 上板的时候，因为有一些 include 语句写在 if VERILATOR 外面，所以综合失败。重新添加 ifdef VERILATOR 之后就好了。